

# Coach11

## Auditoria Técnica Completa

Arquitetura | Segurança | Performance | Qualidade

Data: 18/03/2026

Dominio: coach11.app

Stack: Next.js 16 + React 19 + Supabase + TypeScript

## Índice

---

1. Resumo Executivo
2. Arquitetura UML do Projeto
3. Erros e Problemas Identificados
4. Auditoria de Segurança
5. Análise de Performance e Escalabilidade
6. Refactoring Necessário
7. Smoke Test / Community Test
8. Migração de Domínio (vercel.app -> coach11.app)
9. Plano de Ação por Sprints

# 1. Resumo Executivo

O Coach11 e uma aplicacao web/PWA para gestao de escaloes de futebol em clubes. Esta auditoria avalia o estado actual do projecto em 6 dimensoes: arquitectura, seguranca, performance, qualidade de codigo, testes e preparacao para producao.

## Scorecard Geral

| Area             | Score  | Observacao                                    |
|------------------|--------|-----------------------------------------------|
| Seguranca        | 8.5/10 | Forte - CSP, RLS, sanitizacao, rate limiting  |
| Arquitectura     | 7.5/10 | Solida age_group-first, domain boundaries     |
| Qualidade Codigo | 8/10   | 0 erros TS, 0 any, boa tipagem                |
| Performance      | 6/10   | Todas paginas client-side, sem loading states |
| Testes           | 4/10   | 17 test files, 0 E2E, 0 componentes           |
| Producao-ready   | 6.5/10 | Dominio OK, PWA OK, falta CI/CD robusto       |

PONTOS FORTES: Zero erros TypeScript, zero uso de 'any', security headers robustos, domain boundaries com guardrails automatizados, RPC parametrizado, sanitizacao de HTML, rate limiting implementado.

PONTOS A MELHORAR: 22 paginas todas client-rendered, sem loading.tsx/error.tsx, cobertura de testes insuficiente para escalar, 66% das API routes sem validacao Zod, rate limiting in-memory (nao distribuido).

## 2. Arquitectura UML do Projeto

---

### 2.1 Visao Geral da Estrutura

O projecto segue o Next.js App Router com a seguinte organizacao:

```
src/
  app/      -> Rotas (pages, layouts, API)
  (auth)/   -> Login, registo, convites
  (dashboard)/ -> Area autenticada principal
  api/      -> 62 API routes REST
  public/   -> Paginas publicas por token
  components/ -> 20 modulos de UI organizados por dominio
  lib/      -> Logica de negocio, hooks, utils, services
  types/    -> Definicoes TypeScript (database.ts manual)
```

### 2.2 Diagrama de Componentes

CAMADA DE APRESENTACAO (Client Components)

```
|-- Layout (Sidebar, Navigation, TopBar)
|-- Dashboard (DashboardPage, StatsCards, UpcomingEvents)
|-- Players (PlayerList, PlayerCard, PlayerForm)
|-- Games (GameList, GameDetail, LiveGame, Convocation)
|-- Trainings (TrainingList, AttendanceGrid)
|-- Calendar (CalendarView, EventModal)
|-- Statistics (StatsTable, PDFExport)
|-- Settings (TeamSetup, AccountSettings)
|-- PWA (PWAProvider, InstallPrompt, PushNotifications)
```

CAMADA DE LOGICA (Hooks + Services)

```
|-- useLiveGameState (1668 linhas - necessita refactoring)
|-- useTeamSetup, useGameDetailData, useCalendarData
|-- calendar-events.service.ts (795 linhas)
|-- public-share.ts, rate-limit.ts, auth/beta-access.ts
```

CAMADA DE DADOS (Supabase)

```
|-- Supabase Client (SSR + Browser)
|-- RPC Functions (7 stored procedures)
|-- Row Level Security (RLS policies)
|-- Storage (logos, imagens de eventos)
|-- Auth (email/password + recovery)
```

### 2.3 Modelo de Dados (Entidades Principais)

| Entidade      | Descricao                                          |
|---------------|----------------------------------------------------|
| Profile       | Utilizador autenticado (coordenador, treinador, jo |
| AgeGroup      | Escalao - raiz funcional do sistema (ex: Sub-15)   |
| Player        | Jogador pertencente a um escalao                   |
| AgeGroupStaff | Staff tecnico do escalao (coach, assistant_coach)  |
| Game          | Jogo agendado/live/finalizado                      |
| Convocation   | Convocatoria para jogo (draft/confirmed/closed)    |
| GameEvent     | Evento de jogo (golo, cartao, substituicao)        |
| Training      | Sessao de treino                                   |
| Attendance    | Presenca em treino ou jogo                         |
| Competition   | Liga, taca ou amigavel                             |
| Message       | Mensagem de equipa com tracking de leitura         |
| Notification  | Push notification e in-app                         |
| KitConfig     | Configuracao de equipamentos por kit               |

## 2.4 Relacoes Chave

AgeGroup (1) --> (\*) Player  
 AgeGroup (1) --> (\*) AgeGroupStaff  
 AgeGroup (1) --> (\*) Game  
 AgeGroup (1) --> (\*) Training  
 AgeGroup (1) --> (\*) Competition  
 Game (1) --> (\*) Convocation --> (\*) Player  
 Game (1) --> (\*) GameEvent  
 Training (1) --> (\*) Attendance --> (1) Player  
 Profile (1) --> (\*) AgeGroupStaff  
 Profile (1) --> (1) AgeGroup (as coordinator)

## 2.5 Evolucao Architectural Prevista

FASE ATUAL: age\_group-first (escalao unico por coordenador)

FASE 2 (Proxima): Club-first hierarchy  
 Club --> AgeGroupCategory --> AgeGroup  
 Novo: ClubAdmin, cross-age-group visibility

FASE 3: Multi-tenant  
 Platform --> Club --> AgeGroup  
 Subscricoes, billing, onboarding autonomo

## 3. Erros e Problemas Identificados

---

### 3.1 TypeScript e Lint

- OK** 0 erros TypeScript (tsc --noEmit limpo)
- OK** 0 erros de lint (apenas 3 warnings)
- OK** 0 uso de 'any', @ts-ignore, @ts-expect-error

### 3.2 Warnings a Resolver

- BAIXO** getLineupLabel nao utilizado (summary/page.tsx:110)
- BAIXO** 2x <img> em vez de next/image (staff/page.tsx, ClubLogoUpload.tsx)
- BAIXO** 11 eslint-disable (7 exhaustive-deps + 4 set-state-in-effect)

### 3.3 Problemas de Producao

- MEDIO** Build falha sem SUPER\_COORDINATOR\_EMAIL env var

O ficheiro beta-access.ts faz throw no module load time quando a variavel de ambiente nao existe. Isto impede builds locais e CI sem configuracao completa.

- MEDIO** console.info('[auth.debug]') em PWAProvider.tsx em producao

~5 chamadas de debug logging que executam em cada mudanca de estado de auth e resume do foreground. Devem ser gated por NODE\_ENV === 'development'.

## 4. Auditoria de Seguranca

### 4.1 Autenticacao e Autorizacao

- OK** Todas as API routes verificam `supabase.auth.getUser()`
- OK** Admin routes usam `getSuperUserAccess()` dedicado
- OK** RPC functions parametrizadas (0 vectores de SQL injection)
- OK** Open redirect protegido via `sanitizeNextPath()`
- OK** Password minimo 10 caracteres enforced (Zod server-side)
- OK** PKCE OAuth flow configurado correctamente
- MEDIO** Sem `middleware.ts` - sem gatekeeper centralizado de auth

Se um developer esquecer de adicionar auth check a uma nova API route, fica publicamente acessivel. Recomenda-se adicionar `middleware.ts` com route matcher para `/api/*` e `/(dashboard)/*`.

### 4.2 Sessao e Cookies

- MEDIO** Auth cookie `httpOnly: false` (acessivel a JS client)

O Supabase SSR requer acesso client-side ao cookie, mas isto significa que um XSS poderia roubar o session token. O cookie tem `maxAge` de 400 dias, prolongando a janela de vulnerabilidade.

- OK** `SameSite: lax` configurado (proteccao CSRF basica)
- OK** `secure: true` em producao
- INFO** Sem CSRF token explicito (depende de `SameSite` + `JSON-only`)

### 4.3 Headers de Seguranca

- OK** CSP configurado sem `unsafe-eval`
- OK** `X-Frame-Options: DENY, frame-ancestors 'none'`
- OK** `Strict-Transport-Security: max-age=63072000`
- OK** `Permissions-Policy: camera=(), microphone=(), geolocation=()`
- OK** `X-Content-Type-Options: nosniff`

### 4.4 Input Validation

- MEDIO** 34% das API routes usam Zod (21/62)

41 API routes (~66%) nao tem validacao explicita Zod. Usam validacao manual que e mais propensa a erros. Existe helper `parseBody()` reutilizavel em `lib/http/validate.ts`. Recomenda-se migrar para Zod progressivamente.

### 4.5 Rate Limiting

**MEDIO** Maioria das API routes SEM rate limiting

Rate limiting existe apenas para: invites (5/15min), location (12/min), redeem (10/hr). Falta em: games CRUD, players CRUD, messages, notifications, team, push, competitions. Um atacante autenticado pode inundar a maioria dos endpoints.

**MEDIO** Endpoints publicos sem rate limiting

/api/auth/register e /api/auth/beta-access/check nao tem rate limiting, permitindo brute-force ou enumeracao de emails.

**MEDIO** Rate limiting in-memory (per-instance em serverless)

Em Vercel serverless, cada instancia tem o seu rate limiter independente. Para proteccao distribuida real, migrar para Upstash Redis ou Vercel KV.

## 4.6 Upload de Ficheiros

**MEDIO** Upload de SVG aceite (risco de stored XSS)

O endpoint /api/team/logo aceita SVG, que pode conter JavaScript embebido. Positivo: limite 5MB, whitelist de extensoes, validacao MIME, auth check. Recomendacao: remover SVG do whitelist ou sanitizar server-side. Tambem falta validacao magic-byte (tipo real do ficheiro).

## 4.7 XSS e Sanitizacao

**OK** dangerouslySetInnerHTML com sanitizacao completa**OK** escapeHtml() aplicado ANTES de markdown transforms

## 4.8 Enumeracao de Emails

**BAIXO** beta-access/check revela status de invite por email

O endpoint retorna {allowed, reason} para qualquer email, revelando se tem beta invite, staff invite ou e legacy user. Considerar remover 'reason' ou limitar informacao exposta.

**OK** Register retorna erro generico para 'ja registado' (SEC-06)

## 4.9 RLS e Base de Dados

**OK** RLS extensivo em 18+ migrations Supabase**OK** Policies usam security definer functions**OK** Admin client protegido com 'server-only' import**OK** Cross-club boundary policies RESTRICTIVE

## 4.10 Suite de Testes de Seguranca

**OK** security-fixes.test.ts com 19 testes automatizados

Valida: CSP sem unsafe-eval, password min 10 chars, sem hardcoded secrets, score max cap, correlationId hidden em producao, validacao de nomes. Erros em producao retornam apenas 'Erro interno do servidor' sem detalhes.



## 5. Analise de Performance e Escalabilidade

### 5.1 Renderizacao

**ALTO** 22/22 paginas sao 'use client' - zero SSR

TODAS as paginas do dashboard sao client-rendered. Isto significa:

- Maior bundle JS enviado ao browser
- Sem beneficio de streaming SSR do Next.js
- SEO limitado (menos relevante para dashboard, mas critico para paginas publicas)
- Maior Time to Interactive (TTI)

Recomendacao: Migrar paginas para Server Components com 'use client' apenas nos componentes interactivos. Priorizar dashboard, calendar, statistics.

### 5.2 Loading States e Error Boundaries

**ALTO** 0 ficheiros loading.tsx encontrados

**ALTO** 0 ficheiros error.tsx (apenas global-error.tsx)

Sem loading.tsx, o utilizador ve um ecrã branco durante a navegacao. Sem error.tsx por segmento, erros em sub-paginas crashes toda a app.

Recomendacao: Adicionar loading.tsx com skeletons em (dashboard)/ e cada sub-rota. Adicionar error.tsx em (dashboard)/ e (auth)/.

### 5.3 Caching

**OK** React Query (TanStack) para caching client-side

**OK** unstable\_cache para paginas publicas (ISR)

**OK** Cache-Control headers bem configurados para assets

**MEDIO** Sem caching server-side para API routes autenticadas

### 5.4 Queries de Base de Dados

**ALTO** Convocation route: 10+ queries sequenciais (waterfall)

A route GET /api/games/[id]/convocation faz 10+ queries sequenciais ao Supabase que poderiam ser paralelizadas com Promise.all. Inclui: game, access context, convocations, players, external players, same-day games, kit pieces, checkpoints.

**ALTO** resolveUserTeamContext: 3-5 queries POR REQUEST

Esta funcao corre em CADA request autenticado e no dashboard layout, fazendo 3-5 queries sem qualquer caching. Para 100 utilizadores activos, isto representa 300-500 queries/request ao Supabase. Implementar caching com unstable\_cache ou React Query server-side.

**ALTO** N+1 em messages: getUserById por sender

loadAuthDisplayNamesById() chama auth.admin.getUserById() individualmente por cada sender unico. Com muitos senders, cria N+1 auth lookups.

**ALTO** /api/games e /api/trainings sem paginacao

Ambos retornam TODOS os registos sem limite. A medida que as temporadas acumulam, estas respostas crescem sem controlo. Adicionar cursor pagination.

## 5.5 Bundle e Dependencias

**ALTO** radix-ui umbrella package importa tudo

O package 'radix-ui' (v1.4.3) importa TODOS os primitivos Radix mesmo quando so alguns sao usados. Migrar para packages individuais (@radix-ui/react-dialog, etc.) para tree-shaking efectivo.

**OK** leaflet e jspdf ja usam dynamic import

**MEDIO** posthog-js (~45KB) carregado em todas as paginas

## 5.6 React Query Subutilizado

**ALTO** React Query instalado mas maioria usa raw fetch/useState

A maioria das pages usa fetch() + useState/useEffect em vez de React Query. Perde-se deduplicacao, caching, background refetching e retry automatico. Migrar progressivamente para useQuery/useMutation.

## 5.7 Optimizacoes React

**OK** 111 usos de useMemo/useCallback em 27 ficheiros

**MEDIO** Sem React.memo em list items (ConvocatedRow, etc.)

**MEDIO** Pages com 15+ useState - considerar useReducer

## 5.8 Polling Redundante

**MEDIO** Notifications: polling 60-120s + Supabase Realtime

use-unread-notifications.ts faz polling E subscrive Supabase Realtime. O polling e redundante - remover e confiar apenas no realtime.

## 5.9 Memory Leaks

**OK** Todos event listeners e intervals limpos correctamente

**OK** Supabase realtime channels limpos no cleanup

Nenhum memory leak detectado - boa pratica de cleanup.

## 6. Refactoring Necessario

### 6.1 Ficheiros Grandes (Prioridade Alta)

| Ficheiro                    | Linhas | Accao Recomendada                         |
|-----------------------------|--------|-------------------------------------------|
| useLiveGameState.ts         | 1,668  | Dividir em sub-hooks por responsabilidade |
| games/[id]/summary/page.tsx | 952    | Extrair componentes de sumario            |
| competitions/page.tsx       | 928    | Separar CRUD de display                   |
| team/page.tsx               | 909    | Extrair tabs em componentes               |
| staff/page.tsx              | 840    | Separar lista de formularios              |
| players/page.tsx            | 813    | Extrair PlayerForm e PlayerList           |
| calendar-events.service.ts  | 795    | Dividir por tipo de evento                |
| games/page.tsx              | 680    | Extrair GameCard e GameFilters            |
| LandingPage.tsx             | 664    | Dividir em seccoes                        |
| settings/page.tsx           | 652    | Extrair cada tab em componente            |

### 6.2 Validacao de API Routes

**ALTO** 41 API routes sem validacao Zod (66% do total)

Criar schemas Zod para todas as API routes. Priorizar routes que aceitam POST/PUT/PATCH. Padrao recomendado: `schema.safeParse(body)` no inicio de cada handler.

### 6.3 Tipos de Base de Dados

**MEDIO** database.ts e manual (298 linhas) - risco de dessincronia

Usar 'supabase gen types typescript' para gerar tipos automaticamente. Manter database.ts como facade que re-exporta os tipos gerados.

### 6.4 Camada de Repositorio

**MEDIO** Apenas 3 ficheiros em src/lib/repositories/

57+ ficheiros usam Supabase directamente. Consolidar queries repetidas em repositorios por dominio: PlayerRepository, GameRepository, etc.

### 6.5 Blocos catch {} Vazios

**ALTO** 104 blocos catch {} vazios em todo o codebase

104 blocos catch que engolem erros silenciosamente. Torna o debugging extremamente dificil. Adicionar pelo menos `console.error` ou `toast` em cada um. Priorizar os que estao em componentes client-side e API routes.

### 6.6 API Routes sem respondInternalServerError

**MEDIO** 3 API routes sem error reporting padronizado

Falta respondInternalServerError + Sentry em:

- api/calendar/events/route.ts
- api/invite/redeem/route.ts
- api/waitlist/route.ts

Todas as outras ~58 routes usam o padrao correctamente.

## 6.7 Chamadas Supabase Directas em Pages

**MEDIO** 6 dashboard pages com supabase.from() directo

Pages que devem migrar para hooks/repositories:

- competitions/page.tsx
- dashboard/page.tsx
- join/page.tsx
- messages/page.tsx
- notifications/page.tsx
- settings/page.tsx

## 6.8 Padroes de Data Fetching Inconsistentes

**MEDIO** 3 padroes diferentes de fetch no client-side

O codebase usa 3 padroes diferentes:

1. fetch() directo com res.json().catch(() => ({}))
2. apiFetch<T>() wrapper (lib/http/apiFetch.ts)
3. supabase.from() directo em pages

Recomendacao: Standardizar em apiFetch para client->API, mover Supabase para server components ou camada de repositorio.

## 6.9 eslint-disable Comments

**BAIXO** 11 eslint-disable comments (7 exhaustive-deps)

Revisar cada supressao - muitas podem ser resolvidas com useCallback ou refs. As restantes devem ter comentarios explicativos.

## 6.10 Environment Variables

**MEDIO** 20+ env vars sem .env.example documentado

Variaveis criticas dispersas pelo codebase sem documentacao centralizada. Nota: SUPABASE\_SERVICE\_ROLE\_KEY tem 3 nomes fallback diferentes - consolidar. Criar .env.example com todas as variaveis e descricoes.

## 7. Smoke Test / Community Test

### 7.1 Resultados

- OK** 17 ficheiros de teste, 66 testes unitarios - TODOS PASSAM
- OK** Lint: 0 erros, 3 warnings
- OK** Architecture guard: 313 ficheiros, PASSED
- MEDIO** Build: FALHA sem env vars (SUPER\_COORDINATOR\_EMAIL)

### 7.2 Cobertura de Testes Existente

| Ficheiro                        | Testes    | Area                     |
|---------------------------------|-----------|--------------------------|
| security-fixes.test.ts          | 19 testes | Validacao de seguranca   |
| sanitize-next.test.ts           | 6 testes  | Open redirect protection |
| canonical-app-url.test.ts       | 3 testes  | URL canonica             |
| convocation-editor.test.ts      | Varios    | Editor de convocatorias  |
| live-kickoff.test.ts            | Varios    | Inicio de jogo live      |
| live-persistence.test.ts        | Varios    | Persistencia de estado   |
| public-live/convocation.test.ts | Varios    | Paginas publicas         |
| osm/google-place-id.test.ts     | 8 testes  | Providers de localizacao |

### 7.3 Lacunas Criticas de Testes

- CRITICO** 0 testes E2E (Playwright/Cypress)
- CRITICO** 0 testes de componentes React
- ALTO** 0 testes de integracao para API routes
- MEDIO** Sem .env.example para onboarding de developers
- MEDIO** Sem vitest.config.ts (defaults usados)

### 7.4 Community Test Recomendado

Fluxos criticos que DEVEM ter testes E2E:

1. Login -> Dashboard -> Ver escalao
2. Criar jogo -> Convocatoria -> Confirmar -> Live game
3. Live game -> Golos/Cartoes -> Finalizar -> Sumario
4. Criar treino -> Marcar presencas
5. Convidar staff -> Aceitar convite -> Acesso ao escalao
6. Pagina publica -> Verificar dados visiveis
7. PWA -> Push notification -> Click -> Navegacao

8. Exportar PDF de estadísticas

## 8. Migração de Domínio

---

### 8.1 Estado Atual

**OK** Código-fonte 100% migrado para coach11.app

**BAIXO** 1 referência restante em README.md (exemplo curl)

### 8.2 Ações Necessárias

- Atualizar README.md:53 - curl URL de coach11.vercel.app para coach11.app
- Verificar configuração DNS no Vercel (domínio custom coach11.app)
- Configurar redirect 301 de coach11.vercel.app para coach11.app
- Atualizar Supabase Auth redirect URLs para coach11.app
- Verificar CORS origins no Supabase dashboard
- Atualizar canonical URLs em meta tags (SEO)
- Verificar push notification VAPID\_SUBJECT usa coach11.app
- Atualizar quaisquer webhooks externos (Resend, Sentry, PostHog)

## 9. Plano de Acao por Sprints

---

### Sprint 1: Fundacao e Qualidade (Semana 1-2)

#### Prioridade: Critico

- Criar .env.example com todas as env vars necessarias
- Fix: beta-access.ts - mover throw para runtime, nao module load
- Adicionar loading.tsx em (dashboard)/ com skeleton screens
- Adicionar error.tsx em (dashboard)/ e (auth)/
- Remover getLineupLabel nao utilizada
- Atualizar README.md com dominio coach11.app
- Configurar Playwright para testes E2E
- Criar 3-5 testes E2E para fluxos criticos (login, criar jogo, live game)
- Gating dos console.info debug em PWAProvider.tsx
- Corrigir catch {} vazios mais criticos (API routes e hooks)
- Adicionar respondInternalServerError a 3 API routes em falta
- Consolidar 3 nomes fallback de SUPABASE\_SERVICE\_ROLE\_KEY

### Sprint 2: Performance e Validacao (Semana 3-4)

#### Prioridade: Alto

- Adicionar Zod validation a API routes POST/PUT (priorizar games, players, staff)
- Corrigir restantes catch {} vazios (104 no total)
- Paralelizar queries na convocation route (Promise.all)
- Adicionar paginacao a /api/games e /api/trainings
- Cachear resolveUserTeamContext (corre 3-5 queries por request)
- Corrigir N+1 getUserById em messages route
- Migrar radix-ui umbrella para packages individuais
- Migrar pages para React Query (eliminar raw fetch/useState)
- Remover polling redundante em notifications (manter Realtime)
- Migrar dashboard page para Server Component
- Substituir <img> por next/image em staff/page.tsx e ClubLogoUpload.tsx
- Refactorar useLiveGameState.ts (1668 linhas) em sub-hooks
- Implementar rate limiting distribuido (Upstash Redis)
- Adicionar mais 5-10 testes E2E para fluxos secundarios



## Sprint 3: Refactoring e Arquitectura (Semana 5-6)

### Prioridade: Medio

- Refactorar pages grandes: summary, competitions, team, staff, players (>800 linhas)
- Expandir camada de repositorio (PlayerRepo, GameRepo, TrainingRepo)
- Mover 6 pages com supabase.from() directo para hooks/repositories
- Standardizar data fetching em apiFetch (eliminar 3 padroes diferentes)
- Gerar tipos Supabase automaticamente (supabase gen types)
- Revisar e resolver 7 eslint-disable exhaustive-deps
- Adicionar middleware.ts centralizado para auth (opcional mas recomendado)
- Adicionar testes unitarios para repositories e services

## Sprint 4: Escalabilidade Multi-Clube (Semana 7-8)

### Prioridade: Estrategico

- Modelo de dados: Club > AgeGroupCategory > AgeGroup
- Dashboard de clube com vista panoramica de escaloes
- Role 'Club Admin' com acesso cross-escalao
- Switching de contexto entre escaloes (sidebar dropdown)
- Expandir roles de staff (estagiario, preparador fisico, delegado, etc.)
- Cross-escalao: convocatoria de jogadores de outros escaloes
- Testes E2E para novos fluxos multi-escalao

## Sprint 5: Polish e Producao (Semana 9-10)

### Prioridade: Importante

- Pipeline CI/CD completo (lint + tsc + test + build)
- Monitoring e alertas (Sentry alerts, uptime checks)
- Performance audit com Lighthouse (target: >80 em todas as metricas)
- Documentacao tecnica para novos developers
- Security penetration testing manual
- Load testing para cenarios multi-clube (50+ clubes)
- Portal de pais/encarregados (vista read-only)
- Avaliacao de jogadores (fichas individuais)

## Resumo de Accoes Imediatas (Top 10)

| #  | Severidade | Accao                                              |
|----|------------|----------------------------------------------------|
| 1  | CRITICO    | Criar .env.example e fix beta-access.ts build erro |
| 2  | CRITICO    | Adicionar loading.tsx e error.tsx boundaries       |
| 3  | CRITICO    | Configurar Playwright e criar primeiros testes E2E |
| 4  | ALTO       | Corrigir 104 catch {} vazios (erros silenciosos)   |
| 5  | ALTO       | Adicionar Zod validation a 41 API routes           |
| 6  | ALTO       | Refactorar useLiveGameState.ts (1668 linhas)       |
| 7  | ALTO       | Migrar paginas para Server Components              |
| 8  | MEDIO      | Standardizar data fetching (3 padroes -> 1)        |
| 9  | MEDIO      | Implementar rate limiting distribuido              |
| 10 | MEDIO      | Expandir camada repositório + gerar tipos Supabase |

*Este relatorio foi gerado automaticamente com base na analise estatica do codigo-fonte, execucao de testes, e inspecao de configuracoes. Recomenda-se complementar com testes manuais e penetration testing.*